

데이터센터 환경의 KV Cache 확장 기술 다관점 비교 평가

ITME와 CXL-PIM의 데이터 이동 및 메모리 확장 방식 비교

1.문제 정의 및 대상 기술 선정

1.1 선정 도메인

데이터센터/ 클라우드

1.2 문제 정의

데이터센터에서는 여러 사용자의 요청을 동시에 처리해야 하고, 문맥이 길어질수록 KV Cache도 계속 커진다. 그러다 보면 GPU HBM만으로는 저장 공간이 부족해지고, 동시에 처리할 수 있는 요청 수도 줄어들 수 있다.

KV Cache를 GPU 밖의 메모리나 스토리지로 옮기면 저장 공간은 늘릴 수 있지만, 데이터를 다시 GPU로 가져오는 과정에서 지연이나 대역폭 문제가 생길 수 있다.

따라서 GPU HBM의 용량 한계를 줄이면서도 KV Cache 이동 때문에 성능이 크게 떨어지지 않는 방법이 필요하다.

1.3 선정 기술 및 이유

구분	기술	핵심 접근
HW	ITME	CXL/NVMe 기반 계층형 메모리 확장
HW	CXL-PIM	CXL 확장 + 메모리 근접 연산

ITME와 CXL-PIM은 모두 GPU HBM만으로 KV Cache를 저장하기 어려운 문제를 외부 메모리를 활용해서 해결하려는 HW 기술이다.

ITME는 CXL과 NVMe를 이용해 KV Cache를 저장할 수 있는 공간을 계층적으로 넓히는 방식이다. 반면 CXL-PIM은 메모리를 확장하는 것뿐만 아니라 일부 연산을 메모리 가까이에서 처리해서 GPU와 외부 메모리 사이의 데이터 이동을 줄이는 방향이다.

두 기술이 같은 문제를 다루면서도 해결 방식에는 차이가 있기 때문에, 메모리 공간 확장과 데이터 이동 비용이라는 관점에서 비교하기 적합하다고 판단했다.

2.시스템 설계

2.1 기술 선정 방식

기술은 Human 기반으로 선정한다.

처음부터 Agent에게 기술을 고르게 하기보다는, 먼저 데이터센터에서 발생하는 KV Cache 문제를 정하고 그 문제를 비슷한 방식으로 해결하는 ITME와 CXL-PIM을 직접 선정했다.

이후 Agent는 기술을 다시 고르는 것이 아니라, 선정된 두 기술의 자료를 찾고 각 관점에서 평가하는 역할을 맡는다.

2.2 Agent 구성

Agent	역할	주요 입력	주요 출력	사용 도구
기술 조사 Agent	원문 기반 기술 원리·성능·한계 추출	기술명, 논문	<code>`technical_evidence`</code>	RAG
TRL 평가 Agent	기술 근거와 공개 정보를 TRL 단계에 매핑	<code>`technical_evidence`</code>	<code>`trl_analysis`</code>	State Evidence + Web Search
시장 평가 Agent	제품화·채택·생태 계·시장 성장성 조사	기술명	<code>market_analys is</code>	Web Search
이해관계자 평가 Agent	개발자·도입 기업·HW 업체·산업계 반응 조사	기술명	<code>stakeholder_an alysis</code>	Web Search
도메인 평가 Agent	데이터센터 적용성과 성능·비용·확장성 평가	<code>`technical_evidence`</code>	<code>`domain_analys is`</code>	RAG + State Evidence
평가 종합 Agent	관점별 공통점·차이·Tra de-off 종합	, 4개 Agent 평가 결과, <code>`counter_evide nce`</code> , <code>`conflicts`</code>	<code>synthesis</code>	State

보고서 Agent	최종 평가 보고서 생성	<code>`synthesis`, `references`, `missing_evidence`</code>	<code>`final_report`</code>	State
-----------	--------------	--	-----------------------------	-------

기술 조사 Agent는 원문에서 검증 가능한 기술 사실과 실험 결과만 추출한다. TRL 평가 Agent는 기술 원리를 다시 분석하지 않고, ``technical_evidence``와 제품화·실제 적용 관련 공개 정보를 TRL 단계에 매핑한다. 평가 종합 Agent는 새로운 검색을 수행하지 않고 관점별 결과의 일치·불일치와 Trade-off를 종합한다. 보고서 Agent는 평가를 다시 수행하지 않고 State의 결과를 정해진 목차에 맞게 구성한다.

Evidence 충분성 검사, Query Rewrite, Counter-Evidence 검증, Conflict 분석은 독립 Agent가 아니라 Graph의 제어·검증 Node로 구성한다.

2.3 RAG 적용 대상

기술 조사 Agent

기술 조사 Agent는 ITME와 CXL-PIM 원 논문을 대상으로 RAG를 사용한다.

논문에서 다음 내용을 찾는다.

- 기술이 어떤 방식으로 동작하는지
- KV Cache가 어디에 저장되는지
- 데이터가 어떻게 이동하는지
- 어떤 환경에서 실험했는지
- 주요 성능 결과
- 어떤 기술과 비교했는지
- 논문에서 이야기하는 한계점

단순히 논문 내용을 요약하기보다는, 이후 다른 Agent가 평가할 때 사용할 수 있는 **근거를 뽑는 역할**로 사용한다.

TRL 평가 Agent

기술 조사 Agent가 찾은 논문 근거를 먼저 사용한다. 다만 논문만으로 실제 제품화나 상용화 수준까지 확인하기 어려우면 Web Search를 추가로 사용한다.

TRL 값은 공식적으로 공개된 값이 아니라면 공개된 정보를 기준으로 추정한 값으로 처리한다.

도메인 평가 Agent

데이터센터에서 중요한 Capacity, Latency, Throughput, Bandwidth 같은 항목을 평가하기 위해 RAG를 사용한다.

시장 평가 / 이해관계자 평가 Agent

시장 상황이나 기업 반응은 바뀔 수 있기 때문에 미리 임베딩한 문서보다는 Web Search를 이용한다.

상용화 사례나 기업 도입 여부, 개발자 반응, 산업계 시각 등을 찾아서 평가에 사용한다.

2.4 문서 선정 전략

RAG 문서는 이번에 다루는 데이터센터의 KV Cache 외부 메모리 확장 문제와 직접 관련된 HW 논문 위주로 구성한다.

ITME와 CXL-PIM은 실제 비교 대상이기 때문에 두 기술의 원 논문을 핵심 문서로 사용한다. 기술 구조나 성능 수치, 실험 환경 등은 가능하면 원 논문을 기준으로 확인한다.

InfiniGen은 최종 비교 대상은 아니지만 보조 문서로 넣는다. GPU 밖의 메모리에 KV Cache를 두었을 때 데이터 이동과 오프로딩이 왜 문제가 될 수 있는지 확인하는 데 사용할 수 있기 때문이다.

구분	문서	페이지 수	활용 목적
핵심	ITME: Inference Tiered Memory Expansion with Disaggregated CXL-Hybrid Memories	13	CXL/NVMe 기반 메모리 확장 구조, 성능, 실험 환경, 한계 확인
핵심	Scalable Processing-Near-Memory for 1M-Token LLM Inference: CXL-Enabled KV-Cache Management Beyond GPU Limits	13	CXL-PIM 구조, 데이터 이동 방식, 성능, 실험 조건 및 제약 확인
보조	InfiniGen: Efficient Generative Inference of Large Language Models with Dynamic KV Cache Management	18	GPU 외부 메모리 오프로딩과 데이터 이동 병목의 배경 근거

전체 44페이지로, 요구조건의 전체 200페이지 제한을 충족한다.

2.5 RAG 처리 방식

항목	적용 방식
Loading	PDF 단위로 로드하며 문서명과 원본 페이지 정보를 유지한다.
Chunking	문단과 Section 경계를 기준으로 분할하고, 너무 긴 구간만 추가 분할한다.
Metadata	각 Chunk에 technology, document_id, page, chunk_id를 저장한다.

Dens Retrieval	multilingual-e5-base를 이용해 한국어 질의와 영어 논문의 의미적 관련성을 검색한다.
Sparse Retrieval	BM25를 이용해 CXL, PIM, RDMA, TTFT 등 기술명·약어·지표를 검색한다.
결과 결합	Dense Search와 BM25 결과를 결합하고 중복 Chunk를 제거한다.
Evidence 변환	검색된 Chunk의 주장, 문서명, 페이지, 실험 조건을 Evidence 구조로 변환하여 Agent에 전달한다.
검색 검증	원리·실험 조건·성능·한계에 관한 대표 질문으로 필요한 근거가 검색되는지 확인한다.

chunk Size, overlap, Top-K는 임의로 설정하지 않고 대표 질의의 검색 결과를 확인한 뒤 조정한다.

2.6 Embedding 모델

모델 후보	검토 내용
multilingual-e5-small	다국어 검색이 가능하고 가볍지만, Base보다 모델 규모가 작아 검색 품질 측면에서 우선순위를 낮췄다.
multilingual-e5-base	한국어 질의와 영어 논문을 직접 연결할 수 있고, 200페이지 이하 문서 규모에서 로컬로 사용하기에 구현 부담이 크지 않다.
BGE-M3	긴 입력과 다양한 검색 방식을 지원하지만, 이번 과제 규모에서는 기능과 구현 부담이 상대적으로 크다.

임베딩 모델은 multilingual-e5-base를 사용한다.

Doc Pool의 논문은 영어인데 실제 검색 질문은 한국어로 작성할 수 있기 때문에 한국어와 영어를 같이 처리할 수 있는 모델이 필요하다고 봤다.

또 CXL, PIM, RDMA처럼 정확한 기술명이나 약어 검색이 필요하기 때문에 Dense Retrieval만 사용하지 않고 BM25를 함께 사용하는 Hybrid Retrieval로 구성한다.

3. 평가 관점 및 기준

3.1 4가지 관점 별 평가 기준

관점	평가 대상	평가 기준
기술 성숙도	기술의 현재 개발·검증·상용화 수준	논문/PoC/Prototype/실환경 검증/상용 제품 여부를 근거로 TRL 추정
시장성	시장 채택과 기술 생태계	제품화, 실제 도입, CXL/PIM 생태계, 시장 성장성, 도입 장벽
이해관계자	개발자·도입 기업·HW 업체·산업계 시각	기대 효과, 비용 부담, 호환성, 도입 난이도, 운영상 우려
데이터센터 적용성	대규모 LLM Serving 환경에서의 적용 특성	Capacity, Latency, Throughput, Bandwidth, Scalability, 비용 및 구축 복잡도

3.2 데이터센터 적용성 세부 기준

- Capacity: 얼마나 많은 KV Cache를 저장할 수 있는지 확인한다.
- HBM 절감 효과 : 기존에 GPU HBM에 저장해야 했던 KV Cache를 얼마나 외부로 옮길 수 있는지 본다.
- Latency: 외부 메모리에 접근하면서 발생하는 지연이 어느 정도인지 확인한다. 논문에 TTFT나 TPOT가 있다면 같이 확인한다.
- Throughput: 동시에 많은 요청을 처리할 때 처리량이 어떻게 변하는지 본다.
- Bandwidth / Data Movement: GPU와 외부 메모리 사이에서 얼마나 많은 데이터가 이동하는지, 그 과정에서 대역폭 병목이 생기는지 확인한다.
- Attention 실행 위치 : Attention 연산이 GPU에서 이루어지는지, 메모리 근처에서 처리되는지도 확인한다.
- Scalability: Context 길이나 요청 수가 늘어났을 때 계속 확장할 수 있는지 본다.
- Infrastructure Cost / Complexity : 추가 HW가 필요한지, 기존 GPU 서버에 적용하기 어려운 구조인지 확인한다.

4. State 및 Graph 설계

4.1 State Schema

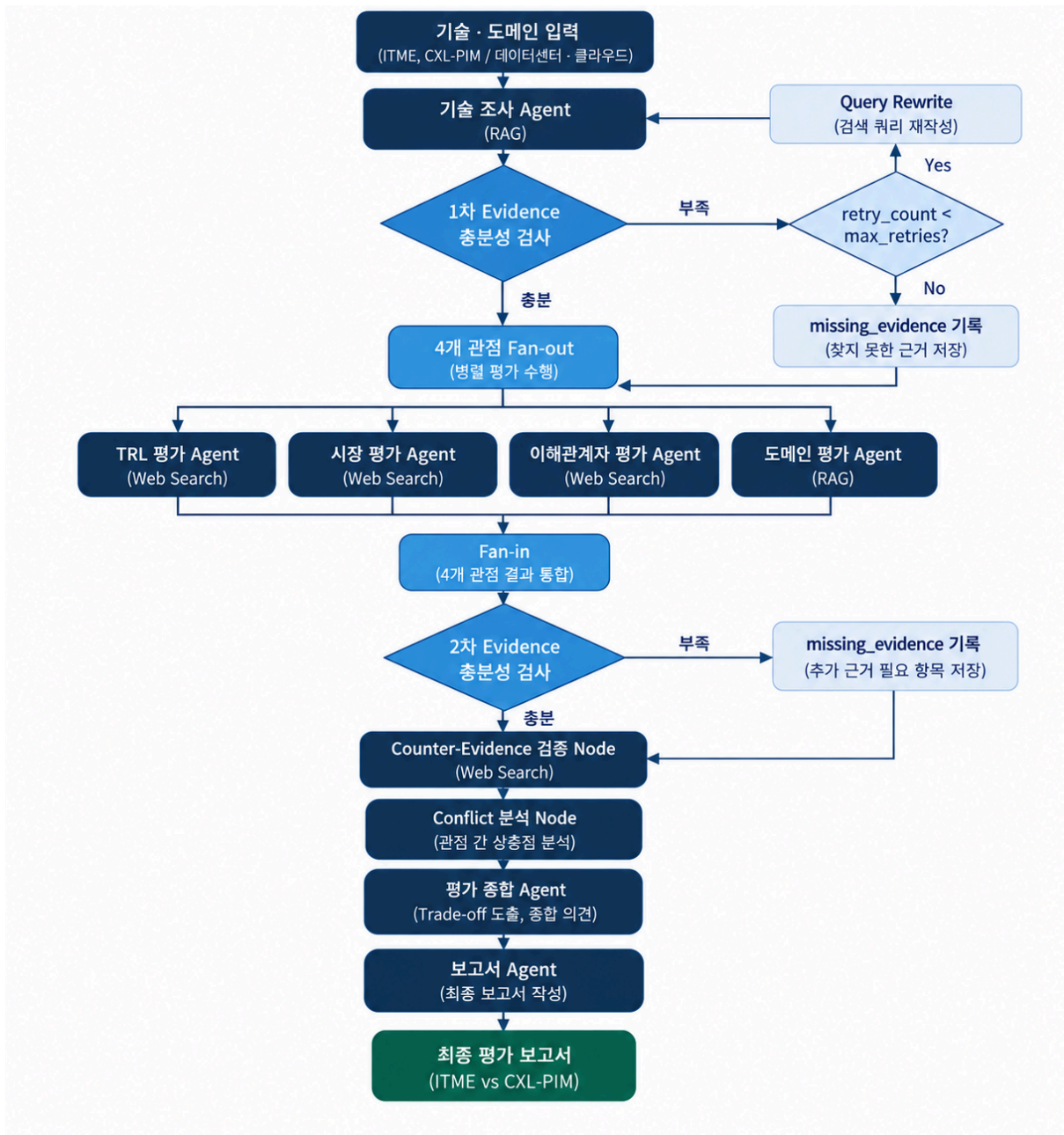
State Key	Type	생성/수정 Agent	사용 Agent	설명
technologies	list	초기 입력	전체	비교 대상 ITME, CXL-PIM

domain	str	초기 입력	Domain Agent	데이터센터·클라우드
technical_evidence	dict	기술 조사 Agent	TRL/Domain/종합	evidence_id 를 key로 기술 원리·성능·실험 조건·한계 근거 저장
market_analysis	dict	시장 평가	종합	시장성 평가 결과
stakeholder_analysis	dict	이해관계자 평가	종합	이해관계자별 시각
domain_analysis	dict	도메인 평가	종합	데이터센터 적용성 평가
trl_analysis	dict	TRL 평가	종합	TRL 평가 결과
missing_evidence	list	1·2차 Evidence 검사 Node	종합 / Report	찾지 못한 근거
retry_count	int	Retrieval	Graph	현재 재검색 횟수
max_retries	int	초기 설정	Graph	최대 재검색 횟수
counter_evidence	dict	Counter-Evidence Node	Conflict/종합	주요 주장별 반대 근거 또는 not_found 상태 저장
conflicts	list	Conflict 분석 Node	종합 / Report	조건·관점·출처에 따른 상충 내용을 저장
synthesis	dict	평가 종합 Agent	Report Agent	공통점, 차이, Trade-off 정리
references	list	Fan-in Node	Report Agent	각 Agent가 사용한 출처를 병합하고 중복 제거
final_report	str	Report Agent	Output	최종 보고서

공통 Evidence 구조

모든 근거에는 evidence_id, technology, perspective, claim, source를 필수로 저장한다. 페이지를 확인할 수 있는 자료는 page를 추가하며, 성능 수치가 포함된 근거는 원문에서 확인된 실험 조건을 experimental_condition에 추가한다. 확인되지 않은 정보는 추정하지 않는다. 각 평가 결과는 사용한 evidence_id를 참조한다.

4.2 Graph 설계



Branch: 1차 Evidence가 충분한지 확인하고, 충분하면 평가를 진행하고 부족하면 Query Rewrite로 보낸다.

Loop: 기술 조사에 필요한 근거가 부족한 경우 최대 재검색 횟수까지 검색을 반복한다.

Fan-out: 기술 조사 이후 TRL, 시장, 이해관계자, 도메인 평가를 병렬로 진행한다.

Fan-in: 4개 Agent의 결과를 다시 모아서 2차 Evidence 검사를 진행한다.

검증: 마지막에는 Counter-Evidence와 Conflict 분석을 통해 한쪽 자료만 보고 결론을 내리지 않도록 한다.

Evidence Gap : 2차 검사에서 찾지 못한 정보는 다시 전체 검색을 반복하지 않고 missing_evidence에 남긴다.

4.3 Evidence 검사 및 재검색

1차 Evidence 검사에서는 기술 조사 결과에 아래 내용이 있는지 확인한다.

- 작동 원리
- KV Cache 저장 위치
- 데이터 이동 방식
- 실험 환경
- 주요 성능 결과
- 비교 Baseline
- 한계점
- 출처

필요한 근거가 부족하면 Query를 다시 작성해 재검색한다. 단, 계속 반복되는 것을 막기 위해 max_retries를 두고 최대 2회까지만 재검색한다.

$retry_count < max_retries \rightarrow$ Query Rewrite 후 재검색

$retry_count \geq max_retries \rightarrow$ 부족한 내용을 missing_evidence에 기록하고 다음 단계로 진행

2차 Evidence 검사는 4개 관점 평가가 끝난 뒤 진행한다. TRL, 시장성, 이해관계자, 도메인 평가에 필요한 근거가 제대로 모였는지 확인한다.

이 단계에서는 다시 전체 Agent를 실행하지 않고, 부족한 내용은 missing_evidence에 남겨 최종 보고서의 한계점에 반영한다.

4.4 검증 및 종합

Counter-Evidence Node에서는 TRL, 시장, 이해관계자, 도메인 평가에서 도출된 기존 주장과 반대되는 결과나 제약 조건이 있는지 Web Search로 확인한다. 반대 근거가 없으면 내용을 따로 생성하지 않고 공개된 반대 근거를 확인하지 못함으로 기록한다. Counter-Evidence 검색은 별도의 반복 Loop을 만들지 않고 주장별 1회만 수행한다.

Counter-Evidence 구조

target_claim, counter_claim, source, page_or_url, status를 저장한다. target_claim에는 검증 대상의 주장을, counter_claim은 확인된 반대 결과 또는 제약 조건을 저장한다. status는 'found' 또는 'not_found'로 제한한다.

Conflict 분석에서는 다음과 같은 부분을 확인한다.

- 성능과 비용 사이의 Trade-off
- 메모리 확장과 데이터 이동량의 관계
- 연구 결과와 실제 상용화 수준의 차이
- 이해관계자별 의견 차이
- 논문별 실험 환경 차이

특히 두 논문의 GPU, 모델, Context Length 등의 조건이 다른 경우에는 성능 수치를 바로 비교하지 않고, 직접 비교가 어렵다는 점 자체를 분석 결과로 남긴다.

마지막으로 평가 종합 Agent가 각 관점의 공통점, 차이, Trade-off를 정리하고 보고서 Agent가 이를 바탕으로 최종 보고서를 생성한다.

5. 보고서 구조 설계

5.1 보고서 작성 목적

특정 기술의 우열을 판정하지 않고, 동일한 KV Cache 문제를 해결하는 두 기술이 기술 성숙도, 시장성, 이해관계자 및 데이터센터 적용 관점에서 어떻게 다르게 평가되는지와 주요 Trade-off를 전달한다.

5.2 보고서 목차 (초안)

SUMMARY

1. 분석 배경 및 문제 정의

1.1 데이터센터 환경의 KV Cache 문제

1.2 분석 목적 및 범위

2. 평가 대상 기술 선정

2.1 ITME

2.2 CXL-PIM

2.3 기술 선정 및 비교 이유

3. 기술 개요

3.1 ITME 핵심 구조와 특성

3.2 CXL-PIM 핵심 구조와 특성

3.3 두 기술의 구조적 차이

4. 다관점 평가

4.1 기술 성숙도

4.2 시장성

4.3 이해관계자

4.4 데이터센터·클라우드 적용성

5. 관점 간 종합 및 시사점

5.1 공통적으로 확인된 사항

5.2 관점에 따라 평가가 엇갈리는 지점

5.3 주요 Trade-off

5.4 종합 의견

6. 분석 한계 및 신뢰성 확보

6.1 공개 정보 기반 분석의 한계

6.2 서로 다른 실험 환경에 따른 직접 비교 한계

6.3 확증편향 방지 및 근거 검증 방법

REFERENCE

SUMMARY는 개요 목록이 아니라 전체 평가 결과의 핵심 요약으로 작성하며 1/2페이지를 넘지 않도록 한다.

REFERENCE에는 최종 보고서에 실제 사용한 자료만 포함한다.

- 논문: 저자(연도). 논문명. 학술지·학회명 또는 arXiv, URL

- 특허: 출원인(연도). 특허명. 특허번호 또는 공개번호, URL

- 웹페이지: 기관명 또는 작성자(발행일). 제목. 사이트명, URL

5.3 보고서 생성 원칙

- 특정 기술을 추천하거나 승자를 결정하지 않음
- 동일 기준에서 두 기술의 특성을 비교
- 실험 환경이 다른 성능 수치를 단순 비교하지 않음
- TRL은 공개 정보 기반 추정임을 표시
- Fact와 이해관계자 Opinion을 구분
- 근거 부족 항목은 추측하지 않고 정보 부족으로 표시
- 상충되는 근거는 제거하지 않고 Trade-off로 정리
- 실제 사용된 자료만 Reference에 포함